Fork, Explore, Commit: Linux Primitives for AI Agents Exploration

Cong Wang[1], Yusheng Zheng[2]

[1]Multikernel Technologies, Inc. · [2]eunomia-bpf / UCSC

Open Source Summit 2026

Agenda

1. **Background**: AI agents as ordinary Linux workloads
2. **Problem**: speculative execution leaves real filesystem and process side effects
3. **Requirements**: what fork-explore-commit needs from the OS
4. **Design**: branch contexts as the abstraction
5. **Implementation**: BranchFS in userspace, `branch()` in the kernel
6. **Evaluation & Status**: latency, demo, limitations, roadmap

What Is an AI Agent?

An **AI agent** is a control loop:

1. **Reason** with an LLM
2. **Act** in a local workspace with tool calls (shell commands, file edits)
3. **Observe** the result and repeat

Examples

Tool class	What it does locally
Claude Code, SWE-agent, OpenHands	edit your repo and run your tests
Aider, Cursor agents	same loop inside an editor
Devin, OpenAI Codex CLI	same loop on a hosted machine

In systems terms: the agent has authority to mutate a workspace.

Agent Exploration and Forking

Agents increasingly try **multiple paths in parallel**, then keep the winner:

Pattern	What it does
Parallel Work	Run N candidates, pick the best
Tree-of-Thoughts	Branch out, prune losers, recurse
Reflexion	Retry on failure with self-critique
Speculate	Race candidates, take first success

Example: agent tries 3 candidate bugfixes on the same repo; commit only the one whose tests pass.

How people do this today

Hack	Why it hurts
`cp -r workspace branch1/`	10 GB monorepo → slow, disk hog
`git stash` per attempt	Misses `node_modules`, build artifacts
Docker container per try	Heavyweight, needs daemon, root
One workspace, retry serially	No parallelism, wall-clock loss
`chroot` + bind mounts	Privileged, racy setup, no atomic commit

What we want

Agents on Linux: Processes with Effects

From `strace`, an agent looks like a developer typing fast:

```
execve("/bin/sh", ["sh", "-c", "pytest -x"], ...)
openat(AT_FDCWD, "src/parser.py", O_WRONLY|O_TRUNC)
write(3, "def parse(s):\n    ...", 4096)
execve("/usr/bin/git", ["git", "apply", "fix.patch"])
execve("/usr/bin/npm", ["npm", "install", "lodash"])
unlink("node_modules/.package-lock.json")
```

What this means for Linux

- They run as **ordinary processes**
- They produce **side effects**: dirty trees, installed packages, build artifacts, modified dotfiles
- Nothing in the kernel knows these processes are "speculative"

> The OS sees a normal Unix workload. The agent's *intent* ("this is one of three things I'm trying") is invisible.

Six Requirements for Agentic Exploration

#	Requirement	Why
R1	**Isolated parallel execution**	Concurrent paths modify same files
R2	**Atomic commit + single-winner**	Apply winner’s changes, invalidate siblings
R3	**Hierarchical nesting**	Tree-of-Thoughts explores sub-variants
R4	**Complete filesystem coverage**	Capture *all* modifications, not just tracked files
R5	**Lightweight, unprivileged, portable**	Sub-ms creation, no root, any FS (ext4, XFS, NFS...)
R6	**Process coordination**	Reliable termination, sibling isolation

No existing Linux mechanism satisfies all six. Let's walk through why.

Filesystem Branching in Linux Today

OverlayFS

```
sudo mount -t overlay overlay \
  -o lowerdir=/repo,upperdir=…,workdir=… \
  /mnt/branch1
```

✓ Per-branch view · ✓ All modifications captured · ✓ Portable

✗ `sudo mount` required (rootless overlay is fragile) ✗ **No commit-to-parent**: `rsync upperdir/ → lowerdir/` skips whiteout deletions ✗ **No sibling invalidation** ✗ Nesting is brittle

Btrfs / ZFS subvolumes

```
$ btrfs subvolume snapshot /repo /repo-b1
$ btrfs subvolume snapshot /repo /repo-b2
```

✓ O(1) creation · ✓ Block-level CoW · ✓ Nested subvolumes first-class

✗ **FS-locked**: your CI runs ext4, your colleague's laptop runs ext4 ✗ **No commit-to-parent**: `btrfs subvolume promote` doesn't exist ✗ **NFS / tmpfs / overlayfs** unsupported ✗ ZFS: `zfs promote` inverts parent ↔ child (wrong shape)

Both come close. Both miss **commit-to-parent**, **sibling invalidation**, and either **unprivileged** or **portable** operation.

Process Isolation in Linux Today

Each primitive does one thing well, none does the whole job:

Primitive	What it gives us	Cost
PID namespace	Reliable kill of all descendants	PID 1 init overhead
Mount namespace	Private view of `/mnt/workspace`	Needed anyway for FS isolation
cgroup v2	Reliable group termination via `cgroup.kill` (5.14+)	Setup, often needs root
`clone3()`	Compose namespaces at process creation	Multi-step, no atomicity
`setpgid` / `setsid`	Process groups	Escapable, child can `setsid()` and leave

The kernel exposes the right **ingredients**, but not the right **combinator**.

The Composition Race

To make one branch in userspace

```
1. Mount the FS branch
2. unshare(CLONE_NEWNS)
3. clone3(NEWPID|NEWNS)
4. Move PID into cgroup
5. Install signal/ptrace fence
   (no primitive, DIY)
```

Each step can fail. Each gap between steps is a **race window**.

The bug between steps 3 and 4

```
parent: clone3() returns PID 12345
parent: ... about to add 12345 to cgroup
12345:  fork() → child 12346
parent: cgroup_add(12345), but 12346 is loose
parent: later, cgroup.kill, 12346 survives
parent: /proc-walk to find it. Maybe.
```

The forked grandchild escapes the cgroup. Reliable cleanup now requires a PID-1 babysitter in each branch, which is the PID-namespace overhead we wanted to avoid.

The Real Problem: Atomic Composition

What we keep finding

Every existing primitive does **one thing well**: but agentic exploration needs **all of them, composed atomically**:

```
┌── FS branch (per-branch upperdir)
│
├── new mount namespace
│
├── new PID namespace (or cgroup)
│
├── signal/ptrace fence vs. siblings
│
└── child PID returned to parent
```

Each step in userspace = a **race window** + a **partial-failure path**.

Precedent: why `clone()` exists

Before `clone()`, you could *almost* build threads from `fork()` + shared memory + signal-based scheduling. But there was no atomic "fork that shares VM".

`clone()` was added because composing those bits in userspace had race windows.

Same argument applies to **fork-branch-fence-commit**. The pieces exist; the combinator does not.

Our claim: **branch contexts** are a new OS abstraction, with two pieces (a filesystem (**BranchFS**) and a syscall (**branch()**)) that together compose the existing primitives atomically.

Branch Contexts: The Abstraction

Definition

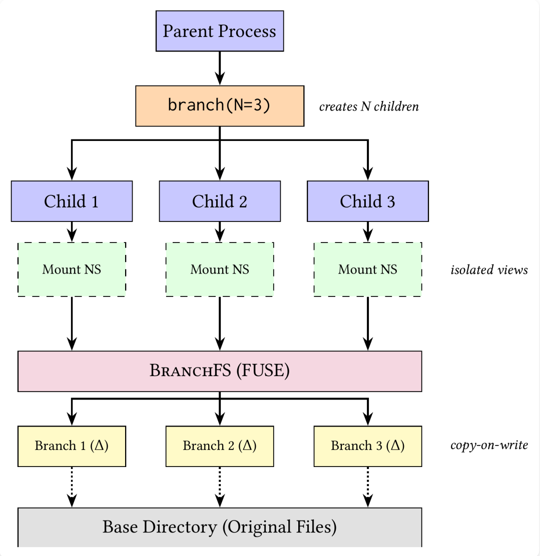
A **branch context** = CoW filesystem view (Δ_i) + confined process group

```
Fork ──▶ Explore ──▶ Commit (winner)
                 └──▶ Abort  (losers)
```

Four core properties

1. **Frozen origin**: parent read-only while branches exist
2. **Parallel isolated execution**: N siblings, fully isolated
3. **First-commit-wins**: siblings auto-invalidated
4. **Nestable**: a branch may fork sub-branches

Architecture



branch() = process coordination (kernel); **BranchFS** = filesystem CoW (FUSE userspace)

BranchFS at a Glance

What it is

- **~3,400 lines of Rust**, FUSE 3
- Userspace daemon: **no root**, no kernel module
- Portable: **ext4, XFS, btrfs, tmpfs, NFS**, any POSIX FS
- MIT / Apache-2.0
- `github.com/multikernel/branchfs`

Using it

```
$ branchfs mount /repo /mnt/work
$ branchctl create /mnt/work feature-a
@feature-a
$ cd /mnt/work/@feature-a
$ vim src/parser.py && make test
$ branchctl commit /mnt/work/@feature-a
```

File-Level Copy-on-Write

- First write to a file → **copy the whole file** into the branch's delta directory
- Subsequent reads and writes to that file are served from the delta
- Unmodified files: pass-through to the base (or an ancestor branch that modified it)

Trade-off vs. block-level CoW

	Block-level (Btrfs)	File-level (BranchFS)
First write of 1 MB file (1 byte changed)	~4 KB	1 MB
Implementation	Kernel-level COW	One FUSE handler
FS dependency	Btrfs-only	Anything

Why this works for agents

- Agent-touched files: source, config, small artifacts
- Typical size: KB to low MB
- Even huge `node_modules` only matters on the first write
- 1 MB copy ≈ 200 μs, still 1000× smaller than an LLM call

Branch Chain Resolution & Tombstones

Looking up a file

For `open("/mnt/work/@b3/src/main.py")`:

```
1. Check @b3's Δ           ← found? serve here
2. Walk ancestors: @b3 → @b1 → base
3. Hit base directory      ← serve here
4. Encounter tombstone?    ← return ENOENT
```

Branches form a chain back to base. Lookups stop at the first hit.

Tombstones handle deletions

Deletion on a branch writes a sentinel:

```
@b3/Δ/.tomb/src/old.py
```

Without tombstones, deleting a file on a branch would let the base copy "reappear" through the chain on next lookup.

Portable by construction: all BranchFS needs from the underlying FS is a writable directory. Branch create = `mkdir`. Branch destroy = `rm -rf`.

Commit: Atomic Promotion to Parent

A commit applies a branch’s delta to its parent in six steps:

```
1. Collect modified files + tombstones from Δ
2. Apply tombstones to parent's Δ      (deletes first)
3. Copy modified files into parent's Δ (then creates)
4. Increment parent's epoch counter    (single atomic op)
5. SIGBUS on siblings' mmap'd regions  (their state is now stale)
6. Sibling's next FUSE op returns -ESTALE
```

Order matters: deletes before creates. Otherwise a delete-then-recreate sequence in the branch could miscompose against an unchanged parent file with the same name.

Abort, Epoch Counter, and Costs

Abort: near-zero cost

- `rm -rf @bN/Δ`
- Siblings remain valid
- Cost proportional only to delta size, not workspace size

Epoch counter: how siblings find out

Each branch carries `(parent_epoch, my_epoch)`.

- Commit to parent: `parent.my_epoch++`
- Sibling’s cached `parent_epoch` no longer matches
- Sibling’s next FUSE op detects mismatch → `-ESTALE`

Costs at a glance

Op	Cost
Create	~300 μs (mkdir)
Commit 1 KB	~317 μs
Commit 1 MB	~2.1 ms
Abort	~315 μs

This is BranchFS's **first-commit-wins primitive**. The `branch()` syscall (later) drives it through ioctls, without changing the semantics.

Performance: Branch, Commit, Abort

Branch Creation: O(1)

Base Size	Latency
100 files	292 µs
1,000 files	317 µs
10,000 files	310 µs

Independent of base size, it's just a `mkdir`.

Commit & Abort

Mod. Size	Commit	Abort
1 KB	317 µs	315 µs
100 KB	514 µs	365 µs
1 MB	2.1 ms	890 µs

Proportional to modification size, not workspace size.

For agents doing LLM calls of **100 ms – 10 s** per step, **sub-millisecond branching is invisible**.

Hardware: AMD Ryzen 5 5500U (6c/12t), 8 GB DDR4, NVMe SSD. Median of 10 trials.

Performance: FUSE Read Throughput

Mode	Read
Native ext4	8.8 GB/s
FUSE (default)	1.7 GB/s
FUSE passthrough	**7.2 GB/s**

82% of native with FUSE 3 passthrough.

50 MB file, 64 KB blocks.

What is FUSE 3 passthrough?

- Added in mainline kernel **6.9**
- Daemon hands the kernel a lower-FD
- Subsequent reads bypass the daemon entirely
- Used by BranchFS for **all unmodified files**
- Modified files still go through the daemon

The "FUSE is slow" reputation comes from the default mode's 19% number. Passthrough closes the gap to ~82% with no application changes.

Demo: A Parallel Agent Run, Start to Finish

```
$ branchfs mount $PWD /mnt/work --base-dir /tmp/branchfs-base &
[branchfs] FUSE 3 passthrough enabled
[branchfs] mounted at /mnt/work, base=/tmp/branchfs-base

$ branchctl create /mnt/work fix-a fix-b fix-c
@fix-a  @fix-b  @fix-c

$ # Agent runs three candidate fixes in parallel
$ (cd /mnt/work/@fix-a && agent run "fix the off-by-one" &> a.log) &
$ (cd /mnt/work/@fix-b && agent run "fix the off-by-one" &> b.log) &
$ (cd /mnt/work/@fix-c && agent run "fix the off-by-one" &> c.log) &
$ wait

$ # Each branch ran tests independently
$ grep -l "passed" {a,b,c}.log
b.log

$ branchctl commit /mnt/work/@fix-b
[branchfs] committed @fix-b → base (epoch 0 → 1)
[branchfs] invalidated siblings: @fix-a, @fix-c

$ branchctl abort /mnt/work/@fix-a /mnt/work/@fix-c
[branchfs] discarded 2 branches
```

Why a Syscall? Userspace Is Not Enough

What BranchFS gives us today

Requirement	Status
R1 isolated views	✓ via delta layers
R2 atomic commit	✓ via epoch counter
R3 nesting	✓ via branch chain
R4 complete FS coverage	✓
R5 unprivileged, portable	✓
R6 process coordination	**✗, userspace cannot do this safely**

Five of six checked. R6 needs the kernel, and that's the next slide.

What the Kernel Has to Do

Five capabilities `branch()` provides, only two are even *possible* from userspace:

Capability	Userspace?
Atomic composition of FS branch + mount NS + process group	Impossible, race windows
Memory branching (page-table CoW for `BR_MEMORY`)	Impossible, kernel only
Reliable process termination of all branch descendants	Cgroups only, needs root
Sibling signal/ptrace fence (siblings can't `kill -9` each other)	PID ns only, PID 1 overhead
Atomic mount-namespace setup with `open_tree` + `move_mount`	Possible but fragile

One syscall composes all five atomically, with kernel-side cleanup on partial failure.

The `branch()` Syscall: Interface

```
long branch(int op, union branch_attr *attr, size_t size);
```

Three operations

op	who calls it
`BR_CREATE`	parent: fork N children, each in its own branch
`BR_COMMIT`	child: apply this branch to parent, kill siblings
`BR_ABORT`	child: discard this branch

`bpf(2)` -style multiplexed union → ABI-extensible.

Composable flags for `BR_CREATE`

Flag	Effect
`BR_FS`	Mount namespace + FS branch (required)
`BR_MEMORY`	Page-table CoW for memory
`BR_ISOLATE`	Signal/ptrace fence between siblings
`BR_CLOSE_FDS`	Close inherited FDs

The `branch()` Syscall: Usage

```
int mnt = open("/mnt/work", O_PATH);
pid_t pids[3];
union branch_attr a = {
  .create = {
    .flags = BR_FS,
    .mount_fd = mnt,
    .n_branches = 3,
    .child_pids = (uintptr_t)pids,
  }
};
int idx = branch(BR_CREATE, &a, sizeof(a));

if (idx == 0) {
  // parent: wait for winner
  while (wait(NULL) > 0);
} else {
  // child: idx is 1, 2, or 3
  if (try_fix(idx)) {
    union branch_attr c = {.commit = {0}};
    int r = branch(BR_COMMIT, &c, sizeof(c));
    if (r == -ESTALE) _exit(1);  // lost the race
  } else {
    union branch_attr ab = {.abort = {0}};
```

Filesystem-Agnostic via Generic ioctls

The contract

`branch()` does **no FS-specific work**. It talks to whichever branching filesystem you've mounted via three generic ioctls:

```
FS_IOC_BRANCH_CREATE  _IO('b', 0)
FS_IOC_BRANCH_COMMIT  _IO('b', 1)
FS_IOC_BRANCH_ABORT   _IO('b', 2)
```

Pattern follows existing generic ioctls:

- `FICLONE` (cross-FS clone)
- `FIEMAP` (extent map)
- `FIDEDUPERANGE` (dedup)

What this means

- **BranchFS** implements the ioctls in its FUSE daemon ✓ today

Flow on `BR_CREATE`

```
                userspace
      branch(BR_CREATE, n=3, mount_fd)
                    │
      ──────────── kernel ────────────
                    │
      ┌─────────────┼───────────────┐
      ▼             ▼               ▼
  vfs_ioctl     vfs_ioctl     vfs_ioctl
  CREATE        CREATE        CREATE
      │             │               │
      ─────── FUSE crossing ───────
                    │
             BranchFS daemon
             (allocate Δ₁, Δ₂, Δ₃)
                    │
      ─────── kernel resumes ───────
                    │
             fork × 3
                    │
      inject branch_id into child's
      pt_regs->ax so syscall returns
```

Working Prototype: Vanilla Linux 6.17 + 3 Patches

The patch series

```
prototype/patches/
├── 0001-branch-add-UAPI-and-internal-headers.patch
├── 0002-branch-implement-syscall-and-ioctls.patch
└── 0003-branch-hook-copy_process-and-do_exit.patch
```

Three logical commits against `v6.17`.

Five-minute build

```
$ ./scripts/build-kernel.sh   # clone + patch + build
$ cargo build --release       # BranchFS
$ make -C test                # test program
$ ./scripts/build-rootfs.sh   # initramfs
$ ./scripts/run-qemu.sh       # boot + run
```

Capability status

Item	Status
`BR_CREATE` / `BR_COMMIT` / `BR_ABORT`	✅
CAS-based first-commit-wins	✅
Sibling SIGKILL	✅
`FS_IOC_BRANCH_*` ↔ BranchFS	✅
`BR_ISOLATE` fence	⚠️ flag accepted, fence not plumbed

Latency: Syscall Path Is Cheap

Steady-state, N=1, QEMU/KVM

Operation	Mean	What's in the path
`BR_CREATE` (parent side)	**61–70 µs**	1× vfs_ioctl to BranchFS via FUSE + 1× `kernel_clone()`
`BR_COMMIT` (child side, steady)	**12–25 µs**	atomic CAS + vfs_ioctl + sibling cleanup (none for N=1)

50 iterations, alternating commit/abort, after the first cold iter.

Comparison to paper's user-side numbers

- Paper measures from CLI invocation → ~300 µs branch creation

What dominates BR_CREATE

```
       70 µs total
       ___________

28 µs  vfs_ioctl(FS_IOC_BRANCH_CREATE)
      , incl. FUSE protocol round-trip
25 µs  kernel_clone()
12 µs  fork_inherit hook, pt_regs override,
       copy_to_user(child_pids)
 5 µs  misc bookkeeping
```

Caveat: these are sanity-check numbers, not paper-quality benchmarks. We'd want: bare metal, multiple base sizes, p99 distributions, comparison vs. `unshare` + `overlayfs` baseline.

The big picture

- LLM step: **100 ms – 10 s**
- `branch(BR_CREATE)`: **70 µs**
- Ratio: **at least 1000:1**

The Python Library: BranchContext

Seven exploration patterns

Pattern	Strategy
Speculate	Race N candidates, first success wins
BestOfN	Run N, commit highest-scoring
Reflexion	Sequential retry with feedback
TreeOfThoughts	Hierarchical nested branches
BeamSearch	Keep top-K at each depth
Tournament	Pairwise elimination via judge
Cascaded	Start with 1, fan out on failure

Each pattern manages branch lifecycle + process isolation

What an agent author writes

```python
from branchcontext import BranchContext

ctx = BranchContext("/mnt/work")

# Best-of-N: try 3 fixes, commit best
result = ctx.best_of_n(
    n=3,
    task=lambda b: try_fix(b),
    score=lambda b: run_tests(b),
)
# Winner is already committed.
# Losers' deltas are gone.
```

The library calls BranchFS today (CLI / ioctls). When `branch()` lands, it switches to the syscall transparently, same Python API.

Today: pure userspace, runs anywhere

- `pip install branchcontext` (PyPI)

Status: What’s Shipping and What’s Not

Shipping today

- **BranchFS**: production-quality FUSE filesystem, Linux + macOS
- **BranchContext**: Python library, 7 patterns, on PyPI
- **branch() prototype**: 3 patches against v6.17, QEMU test harness, passing tests

Honest limitations

- **External side effects** (network, IPC) not rolled back on abort
- **Single-winner only**: no multi-branch merge
- **File-level CoW**: symlinks, hardlinks, FIFOs partial
- `BR_MEMORY` deferred, page-table CoW is real mm work
- **Nested branches** deferred in kernel prototype

Roadmap

Port `branch()` prototype to current mainline (v6.13+), prepare RFC patch series for `linux-kernel@`.

Implement `BR_ISOLATE` signal/ptrace fence, a few hundred lines in `kernel/signal.c` and `kernel/ptrace.c`.

Nested branches in kernel + BranchFS, chain already supports it, just lift the `current->branch != NULL` guard.

Effect gating: buffer network/IPC until commit; agent gateways are a natural interposition point.

Beyond agents

With `n_branches=1`, `branch()` is a generic **try-and-rollback** primitive:

- Package upgrades: try, abort if broken
- System config: try, revert if reboot fails
- Schema migrations: try, roll back on error
- Anything where "do a thing and undo it cleanly if it's bad" is the right shape

> The fork-explore-commit lifecycle is more general than agents.
> Agents are just the loudest current use case.

How to Try It, How to Help

Try it

```
# BranchFS (works today, any Linux)
$ cargo install branchfs
$ branchfs mount /repo /mnt/work

# Python integration
$ pip install branchcontext

# Kernel prototype (5 min on a 24-core box)
$ git clone https://github.com/multikernel/branchcontext
$ cd branchcontext/prototype
$ ./scripts/build-kernel.sh
$ ./scripts/run-qemu.sh
```

Help

What	Where
Bugs / features	GitHub issues
Kernel review	LKML thread (coming soon)

Repos

Project	URL
BranchFS (FUSE FS)	`github.com/multikernel/branchfs`
BranchContext (Python)	`github.com/multikernel/branching`
Paper + prototype	`github.com/multikernel/branchcontext`

License

- BranchFS: MIT / Apache-2.0
- BranchContext: Apache-2.0
- Kernel patches: GPL-2.0 (matches Linux)

Contact

- **Cong Wang**: cwang@multikernel.io

Key Takeaways

1. **AI agents are ordinary Linux processes with extraordinary side effects.** The OS has no abstraction for "this is one of N speculative paths." Today's hacks (cp -r, git stash, containers) don't fit.

2. **Existing primitives don't compose.** OverlayFS, Btrfs, namespaces, cgroups each solve one piece, putting them together in userspace creates race windows.

3. **Branch context = CoW FS view + confined process group.** Fork, explore, commit, with first-commit-wins and nesting. New abstraction, small interface.

4. **BranchFS works today.** FUSE 3, ~3,400 lines of Rust, no root, portable across filesystems. FUSE passthrough closes the perf gap.

5. `branch()` **is a small kernel patch.** Two hooks (copy_process, do_exit), three ioctls, three patches against v6.17. Working prototype in QEMU. RFC coming.

BranchFS: **github.com/multikernel/branchfs**

BranchContext: **github.com/multikernel/branching**

Thank You – Questions?

Cong Wang – cwang@multikernel.io

Yusheng Zheng – yzhen165@ucsc.edu

Open Source Summit 2026

BranchFS: github.com/multikernel/branchfs BranchContext: github.com/multikernel/branching